

JusBill - Comprehensive Interview Guide

This document serves as an exhaustive study guide for technical interviews related to the **JusBill** project. It details the complete system architecture, database schemas, key design trade-offs, and critical interview questions.

1. System Architecture & Tech Stack

JusBill is a full-stack, containerized web application operating on a Client-Server architecture.

- **Frontend:** React.js, Vite, Tailwind CSS, Context API.
- **Backend:** Node.js, Express.js.
- **Database:** MongoDB (Mongoose ORM).
- **Caching:** Redis (via Upstash).
- **DevOps & Deployment:** Docker, Nginx, Google Cloud VM.
- **External Integrations:** Groq API (AI Chat), Cloudinary (Image Hosting), Mailjet/Resend (Emailing), Puppeteer (PDF Generation).

Request Flow:

A client request hits the **Nginx Reverse Proxy** on port 80/443. Nginx serves static assets or proxies API requests to the **Node.js Backend** on port 5000. The backend then verifies JWTs and attempts to fetch data from **Redis**. On a cache miss, it queries **MongoDB**, returns the response, and updates the Redis cache.

2. Database Schemas (MongoDB / Mongoose)

Below are the exact, detailed schemas used in the production environment of JusBill.

A. User Schema

```
const userSchema = new mongoose.Schema({
  name: String,
  email: { type: String, unique: true },
  password: String, // Hashed using bcrypt
  shopId: String,
  role: { type: String, default: "owner" },
  companyName: String,
  mobile: { type: String, unique: true },
  secondaryMobile: String,
  address: String,
  state: String,
  gstin: String,
  profileImage: String,
  qrCode: String,
  bankName: String,
  bankAccount: String,
  ifscCode: String,
  financialSnapshot: {
    lastSales: { type: Number, default: 0 },
    lastReceived: { type: Number, default: 0 },
    lastPending: { type: Number, default: 0 },
    lastSnapshotDate: { type: Date, default: null }
  },
  salesSnapshot: {
    lastSales: { type: Number, default: 0 },
    lastPaid: { type: Number, default: 0 },
    lastUnpaid: { type: Number, default: 0 },
    lastResetDate: { type: Date, default: null }
  }
}, { timestamps: true });
```

B. Invoice Schema

```
const invoiceSchema = new mongoose.Schema({
  shopId: String,
  invoiceId: { type: String }, // Auto-incremented custom ID
  customerId: { type: mongoose.Schema.Types.ObjectId, ref: "Customer" },
  customerName: String,
  customerEmail: String,
  customerAddress: String,
```

```
customerMobile: String,
customerGstin: String,
customerState: String,
items: [{
  productId: mongoose.Schema.Types.ObjectId,
  name: String,
  price: Number,
  qty: Number,
  total: Number
}],
subtotal: Number,
discount: { type: Number, default: 0 },
discountAmount: { type: Number, default: 0 },
gstRate: { type: Number, default: 18 },
gst: Number,
grandTotal: Number,
paymentStatus: { type: String, enum: ["Paid", "Unpaid"], default: "Unpaid" },
description: String
}, { timestamps: true });

// Indexes
invoiceSchema.index({ shopId: 1, createdAt: -1 });
invoiceSchema.index({ shopId: 1, customerName: 1 });
invoiceSchema.index({ shopId: 1, invoiceId: 1 }, { unique: true });
```

C. Product Schema

```
const productSchema = new mongoose.Schema({
  shopId: String,
  name: String,
  price: Number,
  purchasePrice: { type: Number, default: 0 },
  stock: Number,
  unit: String,
  description: String,
  sold: { type: Number, default: 0 },
  isRawMaterial: { type: Boolean, default: false }
}, { timestamps: true });

// Indexes
productSchema.index({ shopId: 1, name: 1 });
productSchema.index({ shopId: 1, price: 1 });
```

D. Customer Schema

```
const customerSchema = new mongoose.Schema({
  shopId: { type: String, required: true },
  name: { type: String, required: true },
  mobile: { type: String, required: true },
  email: { type: String, required: true },
  address: String,
  gstin: String
}, { timestamps: true });

// Indexes
customerSchema.index({ shopId: 1 });
customerSchema.index({ name: 'text', mobile: 'text', email: 'text' });
customerSchema.index({ shopId: 1, mobile: 1 }, { unique: true });
customerSchema.index({ shopId: 1, email: 1 }, { unique: true });
```

3. Design Trade-Offs

1. MongoDB (NoSQL) vs. PostgreSQL/MySQL (SQL)

Trade-off: Invoicing systems typically favor SQL for strict ACID compliance and relational constraints (e.g., preventing deletion of a customer who has invoices). However, MongoDB was selected for its hierarchical document flexibility. An invoice inherently embeds line items (products, quantities). By using an embedded array for items, we avoid expensive JOIN operations on read, which speeds up fetching invoice details. Referential integrity is enforced at the application layer via Mongoose.

2. Puppeteer (Backend) vs. jsPDF (Frontend)

Trade-off: Generating PDFs on the frontend offloads compute to the user's browser, saving server costs. However, frontend libraries often struggle with exact CSS styling, specific fonts, and varying browser implementations, leading to inconsistent outputs. Given that GST invoices require absolute precision and a professional look, Puppeteer (Headless Chrome) was deployed on the backend to guarantee pixel-perfect, reproducible, and text-searchable PDFs, at the cost of higher server memory usage.

3. Redis Caching vs. Direct DB Queries

Trade-off: Querying the database directly ensures absolute consistency (no stale data). Implementing Redis caching introduces complexity in cache invalidation. To optimize the dashboard load time (which aggregates heavy invoice data), Redis was introduced. We trade absolute real-time consistency for a massive performance gain (up to 80% faster reads). Stale data is mitigated by setting short TTLs (Time to Live) and explicitly deleting specific cache keys during POST/PUT/DELETE operations.

4. JWT (Cookies) vs. Session-based Auth

Trade-off: Server-side sessions (stored in Redis or MongoDB) allow instant revocation of a user's access, but require a database lookup on every single API request. JWTs are stateless—the server verifies the signature without a database lookup, allowing massive horizontal scalability. The downside is that JWTs are hard to revoke before expiration. We mitigated security risks by storing the JWT inside an HttpOnly cookie, rendering it immune to Cross-Site Scripting (XSS) attacks.

4. System Design & Interview Questions

Q1. How does JusBill handle high traffic and concurrent database writes, specifically when tracking inventory?

Answer: Inventory tracking is highly susceptible to race conditions. If two cashiers bill the exact same product simultaneously, fetching the stock, subtracting the quantity, and saving it can lead to inaccurate data. In JusBill, instead of doing a standard read-modify-write cycle, the backend utilizes MongoDB's atomic `$inc` operator. When an invoice is finalized, the system executes an `updateOne({ _id: productId }, { $inc: { stock: -quantity } })` command. This ensures the database locks the document during the update, guaranteeing accurate stock deductions even under heavy concurrent load.

Q2. How did you structure your Docker environment, and why is Nginx necessary?

Answer: The application is containerized using Docker Compose with multiple profiles (dev/prod). There are two main containers: the Node.js backend and the Frontend (which uses a multi-stage build). In production, Nginx serves as the reverse proxy. It listens on port 80/443 and serves the compiled React static files directly, which is highly efficient. For any requests starting with `/api`, Nginx acts as a reverse proxy, forwarding the traffic to the Node.js container on port 5000. Nginx is necessary for SSL termination, load balancing, preventing direct exposure of the Node process, and efficiently handling static asset delivery.

Q3. Your backend generates PDFs using Puppeteer. How did you prevent Puppeteer from crashing the server due to high memory usage?

Answer: Puppeteer launches a full instance of Chromium, which is highly memory-intensive. To prevent Out of Memory (OOM) errors on the Google Cloud VM, several optimizations were implemented. First, the browser is launched with flags like `--no-sandbox` and `--disable-dev-shm-usage`. Secondly, instead of launching a completely new browser instance for every single invoice request, the application reuses a single browser instance and simply opens a new Browser Context or Page, which consumes a fraction of the memory.

Q4. How does the AI integration work, and how do you prevent prompt injection or hallucination?

Answer: JusBill integrates the Groq API for rapid LLM inference. When a user asks a question, the backend acts as a middleman. It fetches relevant contextual data from MongoDB (like recent sales figures, low stock items, or customer trends). It then constructs a strict system prompt embedding this context, instructing the AI to only answer

based on the provided business data. By handling this on the backend rather than the frontend client, we keep API keys secure and ensure the LLM receives the exact, unalterable business context needed to prevent hallucinations.

Q5. Describe the database indexing strategy used in JusBill.

Answer: To ensure fast query execution as the dataset grows, strategic indexing was applied. In the *Invoice* collection, a compound index on `{ shopId: 1, createdAt: -1 }` is used because the dashboard heavily queries recent invoices per shop. The *Customer* collection utilizes a text index on `{ name: 'text', mobile: 'text', email: 'text' }` to enable rapid, cross-field search auto-completion when creating an invoice. Unique indexes are also placed on combinations like `{ shopId: 1, invoiceId: 1 }` to prevent duplicate invoice numbers from being generated.